

Web Technology I

Unit 7 – JavaScript

Unit 7. JavaScript [8 Hrs.]

- 7.1. Overview of JavaScript
- 7.2. Advantages of JavaScript
- 7.3. Implementing JavaScript code to HTML page using SCRIPT tag
- 7.4. Variables in JavaScript
- 7.5. JavaScript Data Type-Variant subtypes
- 7.6. JavaScript Functions
- 7.7. Event Handling and JavaScript objects
- 7.8. Document Object Model in JavaScript
 - 7.8.1. Browser Objects and Events
 - 7.8.2. Document Objects and Events
 - 7.8.3. Form Objects and Events
- 7.9. Dialog Box supported by JavaScript.
- 7.10. Form validation

7.1. Overview of JavaScript

https://www.tutorialspoint.com/javascript/javascript_overview.htm

JavaScript was invented by Brendan Eich in 1995, and became an ECMA standard in 1997.

ECMA-262 is the official name of the standard. ECMAScript is the official name of the language.

Note - ECMA originally stood for **European Computer Manufacturers Association**, but now refers to Ecma International, a global non-profit that standardizes information and communication systems, most famously creating the ECMAScript standard that JavaScript is based on.

JavaScript was initially created to “make web pages alive”. (JavaScript is used to create client-side dynamic pages.)

JavaScript is an *object-based scripting language* which is lightweight and cross-platform. JavaScript (JS) is a lightweight, interpreted, or just-in-time compiled programming language. While it is most well-known as the scripting language for Web pages, many non-browser environments also use it, such as Node.js, Apache CouchDB and Adobe Acrobat. JavaScript is a prototype-based, multi-paradigm, single-threaded, dynamic language, supporting object-oriented, imperative, and declarative (e.g. functional programming) styles.

The programs in this language are called *scripts*. They can be written right in a web page's HTML and run automatically as the page loads.

Scripts are provided and executed as plain text. They don't need special preparation or compilation to run. JavaScript is not a compiled language, but it is a translated language. The JavaScript Translator (embedded in the browser) is responsible for translating the JavaScript code for the web browser.

Today, JavaScript can execute not only in the browser, but also on the server, or actually on any device that has a special program called the JavaScript engine.

What is JavaScript ?

JavaScript (js) is a light-weight object-oriented programming language which is used by several websites for scripting the webpages. It is an interpreted programming language that enables dynamic interactivity on websites when applied to an HTML document. It was introduced in the year 1995 for adding programs to the webpages in the Netscape Navigator browser. Since then, it has been adopted by all other graphical web browsers. With JavaScript, users can build modern web applications to interact directly without reloading the page every time. The traditional website uses js to provide several forms of interactivity and simplicity.

Although, JavaScript has no connectivity with Java programming language. The name was suggested and provided in the times when Java was gaining popularity in the market. In addition to web browsers, databases such as CouchDB and MongoDB uses JavaScript as their scripting and query language.

JavaScript was first known as **LiveScript**, but Netscape changed its name to JavaScript, possibly because of the excitement being generated by Java. JavaScript made its first appearance in Netscape 2.0 in 1995 with the name **LiveScript**. The general-purpose core of the language has been embedded in Netscape, Internet Explorer, and other web browsers.

The ECMA-262 Specification defined a standard version of the core JavaScript language.

- JavaScript is a lightweight, interpreted programming language.
- Designed for creating network-centric applications.
- Complementary to and integrated with HTML.
- Open and cross-platform

Assignment –

Features of JS

<https://www.javatpoint.com/javascript-tutorial>

Application of JavaScript

JavaScript is used to create interactive websites. It is mainly used for:

- Client-side validation,

- Dynamic drop-down menus,
- Displaying date and time,
- Displaying pop-up windows and dialog boxes (like an alert dialog box, confirm dialog box and prompt dialog box),
- Displaying clocks etc.

Extra –

Case Sensitivity

JavaScript is a case-sensitive language. This means that the language keywords, variables, function names, and any other identifiers must always be typed with a consistent capitalization of letters.

So, the identifiers **Time** and **TIME** will convey different meanings in JavaScript.

NOTE – Care should be taken while writing variable and function names in JavaScript.

7.2. Advantages of JavaScript

The merits of using JavaScript are –

- **Less server interaction** – You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.
- **Immediate feedback to the visitors** – They don't have to wait for a page reload to see if they have forgotten to enter something.
- **Increased interactivity** – You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.
- **Richer interfaces** – You can use JavaScript to include such items as drag-and-drop components and sliders to give a Rich Interface to your site visitors.

7.3. Implementing JavaScript code to HTML page using SCRIPT tag

JavaScript can be implemented using JavaScript statements that are placed within the `<script>... </script>` HTML tags in a web page.

You can place the `<script>` tags, containing your JavaScript, anywhere within your web page, but it is normally recommended that you should keep it within the `<head>` tags.

The `<script>` tag alerts the browser program to start interpreting all the text between these tags as a script. A simple syntax of your JavaScript will appear as follows.

```
<script ...>
  JavaScript code
</script>
```

In more detail

JavaScript segment will look like –

```
<script language = "javascript" type = "text/javascript">
  JavaScript code
</script>
```

The script tag takes two important attributes –

- **Language** – This attribute specifies what scripting language you are using. Typically, its value will be javascript. Although recent versions of HTML (and XHTML, its successor) have phased out the use of this attribute.
- **Type** – This attribute is what is now recommended to indicate the scripting language in use and its value should be set to "text/javascript".

Example 1–

If you need a script to run as the page loads so that the script generates content in the page, then the script goes in the <body> portion of the document. In this case, you would not have any function defined using JavaScript.

```
<html>
<body>
  <h2>Welcome to JavaScript</h2>
  <script language = "javascript" type = "text/javascript">
    document.write("Hello World! By JS");
  </script>
</body>
</html>
```

O/P -

Welcome to JavaScript

Hello World! By JS

Example 2 –

If you want to have a script run on some event, such as when a user clicks somewhere, then you will place that script in the head.

```
<html>
<head>
  <script type = "text/javascript">
    function sayHello() {
      alert("Hello World");
    }
  </script>
</head>

<body>
  <input type = "button" onclick = "sayHello()" value = "Say Hello" />
</body>
</html>
```

O/P

Click here for the result

Say Hello

Extra –

External scripts

If we have a lot of JavaScript code, we can put it into a separate file.

Script files are attached to HTML with the `src` attribute:

```
<script src="/path/to/script.js"></script>
```

Here, `/path/to/script.js` is an absolute path to the script from the site root. One can also provide a relative path from the current page. For instance, `src="script.js"`, just like `src="./script.js"`, would mean a file `"script.js"` in the current folder.

We can give a full URL as well. For instance:

```
<script  
src="https://cdnjs.cloudflare.com/ajax/libs/lodash.js/4.17.11/lodash.js  
></script>
```

To attach several scripts, use multiple tags:

```
<script src="/js/script1.js"></script>  
<script src="/js/script2.js"></script>  
...
```

Example –

index.html

```
<html>  
<head>  
<script type="text/javascript" src="message.js"></script>  
</head>  
<body>  
<p>Welcome to JavaScript</p>  
<form>  
<input type="button" value="click" onclick="msg()"/>  
</form>  
</body>  
</html>
```

message.js

```
function msg(){  
    alert("Hello Javatpoint");  
}
```

Extra –

JavaScript Output

https://www.w3schools.com/js/js_output.asp

JavaScript can "display" data in different ways:

- Writing into an HTML element, using `innerHTML`.
- Writing into the HTML output using `document.write()`.
- Writing into an alert box, using `window.alert()`.
- Writing into the browser console, using `console.log()`.

using `innerHTML` - `document.getElementById(id)` method.

```
<p id="demo"></p>
```

```
<script>
document.getElementById("demo").innerHTML = 5 + 6;
</script>
```

Changing the `innerHTML` property of an HTML element is a common way to display data in HTML.

using `document.write()` -

For testing purposes, it is convenient to use `document.write()`:

```
<script>
document.write(5 + 6);
</script>
```

Using `document.write()` after an HTML document is loaded, will **delete all existing HTML**. The `document.write()` method should only be used for testing.

using `window.alert()`

You can use an alert box to display data.

```
<script>
window.alert(5 + 6);
</script>
```

You can skip the `window` keyword.

```
<script>
alert(5 + 6);
</script>
```

using `console.log()`

For debugging purposes, you can call the `console.log()` method in the browser to display data.

```
<script>
console.log(5 + 6);
</script>
```

7.4. Variables in JavaScript

<https://javascript.info/variables>

Variables are Containers for Storing Data.

A **JavaScript variable** is simply a name of storage location.

There are some rules while declaring a JavaScript variable (also known as identifiers).

All JavaScript **variables** must be **identified** with **unique names**.

These unique names are called **identifiers**.

Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).

The general rules for constructing names for variables (unique identifiers) are:

- Names can contain letters(a to z or A to Z), digits(0 to 9), underscores(_), and dollar signs(\$).
- Names must begin with a letter. Eg – num, num1, etc.
- Names can also begin with \$ and _. Eg - \$num, _num, etc
- Names are case sensitive (a and A are different variables).
- Reserved words (like JavaScript keywords) cannot be used as names. Eg – char, if, else, do, switch, for, etc.

Variable Declaration and initialization

JavaScript Variables can be declared in 4 ways:

- Automatically
- Using `var` (old way) (Function-scoped or global-scoped)
- Using `let` (Block-scoped (limited to `{ }` block))
- Using `const`

Note -

The `var` keyword was used in all JavaScript code from 1995 to 2015.
The `var` keyword should only be used in code written for older browsers.

The `let` and `const` keywords were added to JavaScript in 2015.

Assignment

Var vs let

Feature 	var	let
Scope	Function-scoped or global-scoped	Block-scoped (limited to <code>{ }</code> block)
Re-declaration	Can be re-declared within the same scope	Cannot be re-declared within the same scope
Hoisting	Hoisted to the top of its scope and initialized with <code>undefined</code>	Hoisted, but not initialized (results in a <code>ReferenceError</code> if accessed before declaration)
Modern Practice	Older, generally discouraged in modern JS	Modern, preferred practice for mutable variables

// How to create variables:

```
var x;  
let y;
```

// How to use variables:

```
x = 5;  
y = 6;  
let z = x + y;
```

Example –

Automatically

```
x = 5;  
y = 6;  
z = x + y;
```

Here,

`x`, `y`, and `z` are undeclared variables.

They are automatically declared when first used. **Note** - It is considered good programming practice to always declare variables before use.

```
<!DOCTYPE html>  
<html>  
<body>  
<h1>JavaScript Variables</h1>  
  
<p>In this example, x, y, and z are undeclared.</p>  
<p>They are automatically declared when first used.</p>  
  
<p id="demo"></p>  
  
<script>  
x = 5;  
y = 6;  
z = x + y;  
document.getElementById("demo").innerHTML = "The value of z is: " + z;  
</script>  
  
</body>  
</html>
```

O/P -

JavaScript Variables

In this example, x, y, and z are undeclared.

They are automatically declared when first used.

The value of z is: 11

Using var

```
var x = 5;  
var y = 6;  
var z = x + y;
```

```
<!DOCTYPE html>  
<html>  
<body>  
  
<script>  
var x = 5;  
var y = 6;  
var z = x + y;  
document.write(z);  
//document.write("The value of z is: " + z);  
</script>  
  
</body>  
</html>
```

O/P

11

Using let

```
let x = 5;  
let y = 6;  
let z = x + y;
```

```
<!DOCTYPE html>  
<html>  
<body>  
  
<script>  
let x = 5;  
let y = 6;  
let z = x + y;  
alert(z);  
//alert("The value of z is: " + z);  
</script>  
  
</body>  
</html>
```

Using const

```
const x = 5;  
const y = 6;  
const z = x + y;
```

```

<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Variables</h1>

<p>In this example, x, y, and z are variables.</p>

<p id="demo"></p>

<script>
const x = 5;
const y = 6;
const z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>

</body>
</html>

```

O/P

JavaScript Variables

In this example, x, y, and z are variables.

The value of z is: 11

Types of variables/ (Variable Scope)

There are two types of variables in JavaScript :

1. local variable and
2. global variable

The scope of a variable is the region of your program in which it is defined.

local variable

A JavaScript local variable is declared inside block or function. It is accessible within the function or block only. (A local variable will be visible only within a function where it is defined. Function parameters are always local to that function.) For example:

```

<script>
function abc(){
var x=10;//local variable
}
</script>

```

Or,

```

<script>
If(10<13){
var y=20;//JavaScript local variable
}
</script>

```

global variable

A **JavaScript global variable** is accessible from any function. A variable i.e. declared outside the function or declared with window object is known as global variable. (A global variable has global scope which means it can be defined anywhere in your JavaScript code.)

For example:

```
<script>
var data=200;//global variable
function a(){
  document.writeln(data);
}
function b(){
  document.writeln(data);
}
a();//calling JavaScript function
b();
</script>
```

Note –

Within the body of a function, a local variable takes precedence over a global variable with the same name. If you declare a local variable or function parameter with the same name as a global variable, you effectively hide the global variable.

```
<html>
<body onload = checkscope()>
  <script type = "text/javascript">

    var myVar = "global"; // Declare a global variable
    function checkscope() {
      var myVar = "local"; // Declare a local variable
      document.write(myVar);
    }

  </script>
</body>
</html>
```

O/P

local

7.5. JavaScript Data Type-Variant subtypes

<https://javascript.info/types>

https://www.w3schools.com/js/js_datatypes.asp

JavaScript provides different **data types** to hold different types of values. There are two types of data types in JavaScript.

1. Primitive data type
2. Non-primitive (reference) data type

JavaScript is a **dynamic type language**, means you don't need to specify type of the variable because it is dynamically used by JavaScript engine. You need to use **var** here to specify the data type. It can hold any type of values such as numbers, strings etc. For example:

```
var a=40; //holding number
var b="Rahul"; //holding string
```

JavaScript primitive data types

There are five types of primitive data types in JavaScript. They are as follows:

Data Type	Description
String	represents sequence of characters e.g. "hello"
Number	represents numeric values e.g. 100
Boolean	represents boolean value either false or true
Undefined	represents undefined value
Null	represents null i.e. no value at all

- **Numbers**, eg. 123, 120.50 etc.
- **Strings** of text e.g. "This text string" etc.
- **Boolean** e.g. true or false.

```
// Numbers:
let length = 16;
let weight = 7.5;

// Strings:
let color = "Yellow";
let lastName = "Johnson";

// Booleans
let x = true;
let y = false;

//boolean
<script>
```

```

let isGreater = 4 > 1;

alert( isGreater ); // true (the comparison result is "yes")
</script>

//string

<script>

let name = "John";

// embed a variable
alert( `Hello, ${name}!` ); // Hello, John!

// embed an expression
alert( `the result is ${1 + 2}` ); // the result is 3
</script>

//undefined - The meaning of undefined is "value is not assigned".

<script>

let age;

alert(age); // shows "undefined"
</script>

```

Note – JavaScript does not make a distinction between integer values and floating-point values. All numbers in JavaScript are represented as floating-point values. JavaScript represents numbers using the 64-bit floating-point format defined by the IEEE 754 standard.

Extra –

Most programming languages have many number types:

Whole numbers (integers):

byte (8-bit), short (16-bit), int (32-bit), long (64-bit)

Real numbers (floating-point):

float (32-bit), double (64-bit).

**Javascript numbers are always one type:
double (64-bit floating point).**

JavaScript also defines two trivial data types, **null** and **undefined**, each of which defines only a single value.

JavaScript non-primitive data types

The non-primitive data types are as follows:

Data Type	Description
-----------	-------------

Object	represents instance through which we can access members
Array	represents group of similar values
RegExp	represents regular expression

JavaScript supports a composite data type known as **object**.

Assignment –

Why JavaScript is untyped language? Or dynamically typed language?

JavaScript is **untyped** language. This means that a JavaScript variable can hold a value of any data type. Unlike many other languages, you don't have to tell JavaScript during variable declaration what type of value the variable will hold. The value type of a variable can change during the execution of a program and JavaScript takes care of it automatically.

JavaScript is a dynamically-typed language, which means that the type of a variable can change at runtime.

Extra –

The typeof Operator

You can use the JavaScript **typeof** operator to find the type of a JavaScript variable.

The **typeof** operator returns the type of a variable or an expression:

Example 1 –

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Operators</h1>
<h2>The typeof Operator</h2>
<p>The typeof operator returns the type of a variable or an expression.</p>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
typeof 0 + "<br>" +
typeof 314 + "<br>" +
typeof 3.14 + "<br>" +
typeof (3) + "<br>" +
typeof (3 + 4);
</script>

</body>
</html>
```

O/P

JavaScript Operators

The typeof Operator

The typeof operator returns the type of a variable or an expression.

```
number
number
number
number
number
```

Example 2 –

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Operators</h1>
<h2>The typeof Operator</h2>
<p>The typeof operator returns the type of a variable or an expression.</p>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
typeof "" + "<br>" +
typeof "John" + "<br>" +
typeof "John Doe";
</script>
</body>
</html>
```

O/P

JavaScript Operators

The typeof Operator

The typeof operator returns the type of a variable or an expression.

```
string
string
string
```

Example 3 -

```
<script>
"use strict";

let name = "John";

// embed a variable
alert( `Hello, ${name}!` ); // Hello, John!

// embed an expression
alert( `the result is ${1 + 2}` ); // the result is 3
</script>
```

Example 4 –

```
<script>
"use strict";

let isGreater = 4 > 1;

alert( isGreater ); // true (the comparison result is "yes")
</script>
```

An assignment without `use strict`

Normally, we need to define a variable before using it. But in the old times, it was technically possible to create a variable by a mere assignment of the value without using `let`. This still works now if we don't put `use strict` in our scripts to maintain compatibility with old scripts.

```
1 // note: no "use strict" in this example
2
3 num = 5; // the variable "num" is created if it didn't exist
4
5 alert(num); // 5
```

This is a bad practice and would cause an error in strict mode:

```
1 "use strict";
2
3 num = 5; // error: num is not defined
```

7.6. JavaScript Functions

<https://javascript.info/function-basics>

Quite often we need to perform a similar action in many places of the script.

For example, we need to show a nice-looking message when a visitor logs in, logs out and maybe somewhere else.

Functions are the main "building blocks" of the program. They allow the code to be called many times without repetition. A JavaScript function is a block of code designed to perform a particular task. A JavaScript function is executed when "something" invokes it (calls it).

A function is a group of reusable code which can be called anywhere in your program. This eliminates the need of writing the same code again and again. It helps programmers in writing modular codes. Functions allow a programmer to divide a big program into a number of small and manageable functions.

Like any other advanced programming language, JavaScript also supports all the features necessary to write modular code using functions. You must have seen functions like **alert()** and **write()** in the earlier. We were using these functions again and again, but they had been written in core JavaScript only once.

Assignment –

Advantages of using Function?

There are mainly two advantages of JavaScript functions.

1. **Code reusability:** We can call a function several times so it save coding.
2. **Less coding:** It makes our program compact. We don't need to write many lines of code each time to perform a common task.

Types of Functions

There are mainly two types of functions.

- 1) Built- in Functions
- 2) User-defined functions

Built-in functions

→ like `alert(message)`, `prompt(message, default)` and `confirm(question)`, `write()`, etc

let's see a couple of functions to interact with the user: `alert`, `prompt` and `confirm`.

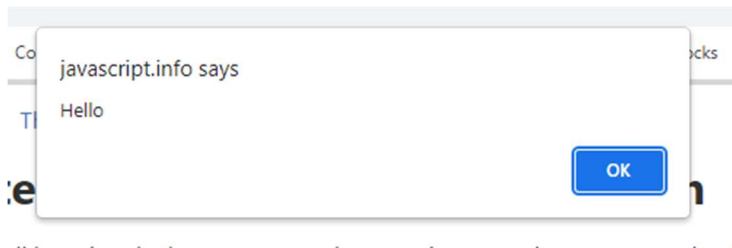
alert

→ It shows a message and waits for the user to press "OK".

For example:

```
<script>
alert("Hello");
</script>
```

The mini-window with the message is called a *modal window*. The word "modal" means that the visitor can't interact with the rest of the page, press other buttons, etc, until they have dealt with the window. In this case – until they press "OK".



prompt

→ shows a message asking the user to input text. It returns the text or, if Cancel button or `Esc` is clicked, `null`.

The function `prompt` accepts two arguments:

```
result = prompt(title, [default]);
```

It shows a modal window with a text message, an input field for the visitor, and the buttons OK/Cancel.

title

The text to show the visitor.

default

An optional second parameter, the initial value for the input field.

The square brackets in syntax [...]

The square brackets around `default` in the syntax above denote that the parameter is optional, not required.

The visitor can type something in the prompt input field and press OK. Then we get that text in the `result`. Or they can cancel the input by pressing Cancel or hitting the `Esc` key, then we get `null` as the `result`.

The call to `prompt` returns the text from the input field or `null` if the input was canceled.

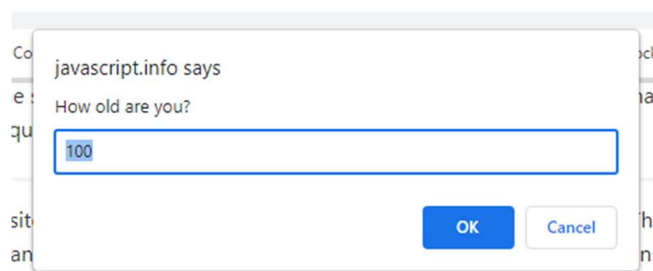
Example –

```
<script>

let age = prompt('How old are you?', 100);

alert(`You are ${age} years old!`); // You are 100 years old!

</script>
```



confirm

→ shows a message and waits for the user to press "OK" or "Cancel". It returns `true` for OK and `false` for Cancel/`Esc`.

The syntax:

```
result = confirm(question);
```

The function `confirm` shows a modal window with a question and two buttons: OK and Cancel.

The result is `true` if OK is pressed and `false` otherwise.

For example:

```
<script>

let isBoss = confirm("Are you the boss?");

alert( isBoss ); // true if OK is pressed

</script>
```



User-defined functions

JavaScript allows us to write our own functions as well.

Function Definition/Declaration

Before we use a function, we need to define it. The most common way to define a function in JavaScript is by using the **function** keyword, followed by a unique function name, followed by parentheses **()** - parentheses may include a list of parameter names separated by commas (that might be empty), and a statement block surrounded by curly braces **{}**.

Syntax

The basic syntax is shown here.

```
<script type = "text/javascript">

    function functionname(parameter-list) {
        statements
    }

</script>
```

OR

```
function functionname(parameter1, parameter2, parameter3) {
    // code to be executed
}
```

Note -

Declared functions are not executed immediately. They are "saved for later use", and will be executed later, when they are invoked (called upon).

Example

It defines a function called sayHello that takes no parameters -

```
<script type = "text/javascript">

    function sayHello() {
        alert("Hello there");
    }

</script>
```

Calling a Function

To invoke a function somewhere later in the script, you would simply need to write the name of that function.

Example -

```
<html>
<head>
    <script type = "text/javascript">
        function sayHello() {
```

```

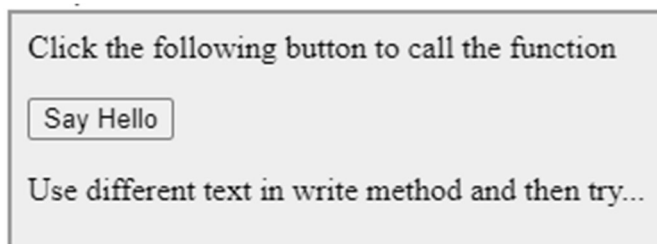
    document.write ("Hello there!");
}
</script>

</head>

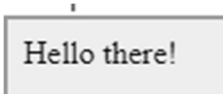
<body>
<p>Click the following button to call the function</p>
<form>
  <input type = "button" onclick = "sayHello()" value = "Say Hello">
</form>
<p>Use different text in write method and then try...</p>
</body>
</html>

```

Output -



After we click "Say Hello" button -



Example of Function -

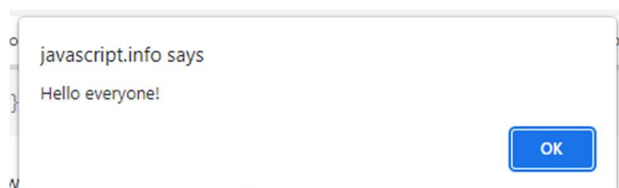
```

<!DOCTYPE html>
<script>

function showMessage() {
  alert( 'Hello everyone!' );
}

showMessage();
showMessage();
</script>

```



Function Parameters

Till now, we have seen functions without parameters. But there is a facility to pass different parameters while calling a function. These passed parameters can be captured inside the function and any manipulation can be done over those parameters. A function can take multiple parameters separated by comma.

Example

We have modified our **sayHello** function here. Now it takes two parameters.

```
<html>
<head>
  <script type = "text/javascript">
    function sayHello(name, age) {
      document.write (name + " is " + age + " years old.");
    }
  </script>
</head>

<body>
  <p>Click the following button to call the function</p>
  <form>
    <input type = "button" onclick = "sayHello('Zara', 7)" value = "Say Hello">
  </form>
  <p>Use different parameters inside the function and then try...</p>
</body>
</html>
```

Output

Click the following button to call the function

Say Hello

Use different parameters inside the function and then try...

After we click “Say Hello” button -

Zara is 7 years old.

Example -

```
<!DOCTYPE html>
<script>
"use strict";

function showMessage(from, text) { // parameters: from, text
  alert(from + ': ' + text);
}

showMessage('Ann', 'Hello!'); // Ann: Hello! (*)
showMessage('Ann', "What's up?"); // Ann: What's up? (**)
</script>
```

Example -

```
<script>
function getcube(number){
  alert(number*number*number);
```

```

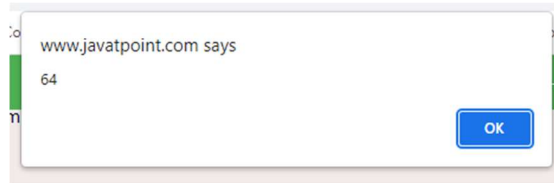
}
</script>
<form>
<input type="button" value="click" onclick="getcube(4)"/>
</form>

```

O/P



After we click "click" button -



The return Statement

A JavaScript function can have an optional **return** statement. This is required if you want to return a value from a function. This statement should be the last statement in a function.

For example, you can pass two numbers in a function and then you can expect the function to return their multiplication in your calling program.

Example

It defines a function that takes two parameters and concatenates them before returning the resultant in the calling program.

```

<html>
<head>
<script type = "text/javascript">
function concatenate(first, last) {
    var full;
    full = first + last;
    return full;
}
function secondFunction() {
    var result;
    result = concatenate("Zara", 'Ali');
    document.write (result );
}
</script>
</head>

<body>
<p>Click the following button to call the function</p>
<form>
<input type = "button" onclick = "secondFunction()" value = "Call Function">
</form>
<p>Use different parameters inside the function and then try...</p>
</body>
</html>

```

Output

Click the following button to call the function

Call Function

Use different parameters inside the function and then try...

After we click "Call Function" button -

ZaraAli

Example -

```
<script>
function getInfo(){
return "hello javatpoint! How r u?";
}
</script>
<script>
document.write(getInfo());
</script>
```

Example -

```
<!DOCTYPE html>
<script>
"use strict";

function sum(a, b) {
    return a + b;
}

let result = sum(1, 2);
alert( result ); // 3
</script>
```

Example -

```

<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Functions</h1>

<p>Call a function which performs a calculation and returns the result:</p>

<p id="demo"></p>

<script>
let x = myFunction(4, 3);
document.getElementById("demo").innerHTML = x;

function myFunction(a, b) {
    return a * b;
}
</script>

</body>
</html>

```

O/P

JavaScript Functions

Call a function which performs a calculation and returns the result:

12

7.7. Event Handling and JavaScript objects

<https://www.javatpoint.com/javascript-events>

https://www.w3schools.com/js/js_events.asp

JavaScript Events

The change in the state of an object is known as an **Event**. In html, there are various events which represents that some activity is performed by the user or by the browser. When javascript code is included in HTML, js react over these events and allow the execution. This process of reacting over the events is called **Event Handling**. Thus, js handles the HTML events via **Event Handlers**.

For example, when a user clicks over the browser, add js code, which will execute the task to be performed on the event.

Here are some others examples of HTML events:

- An HTML web page has finished loading
- An HTML input field was changed
- An HTML button was clicked

Often, when events happen, you may want to do something.

For Example –

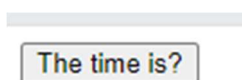
```
<!DOCTYPE html>
<html>
<body>

<button onclick="document.getElementById('demo').innerHTML=Date()">The time is?
</button>

<p id="demo"></p>

</body>
</html>
```

O/P



When you click time will appear like below.

Mon Jun 12 2023 17:00:33 GMT+0545 (Nepal Time)

In the above example, an **onclick** attribute (with code), is added to a **<button>** element.

We can also write above example as –

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Events</h2>
<p>Click the button to display the date.</p>

<button onclick="displayDate()">The time is?</button>

<script>
function displayDate() {
    document.getElementById("demo").innerHTML = Date();
}
</script>

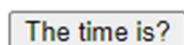
<p id="demo"></p>

</body>
</html>
```

O/P

JavaScript HTML Events

Click the button to display the date.



Some of the HTML events and their event handlers are:

Mouse events:

Event Performed	Event Handler	Description
click	onclick	When mouse click on an element
mouseover	onmouseover	When the cursor of the mouse comes over the element
mouseout	onmouseout	When the cursor of the mouse leaves an element
mousedown	onmousedown	When the mouse button is pressed over the element
mouseup	onmouseup	When the mouse button is released over the element
mousemove	onmousemove	When the mouse movement takes place.

Keyboard events:

Event Performed	Event Handler	Description
Keydown & Keyup	onkeydown onkeyup	& When the user press and then release the key

Form events:

Event Performed	Event Handler	Description
focus	onfocus	When the user focuses on an element
submit	onsubmit	When the user submits the form
blur	onblur	When the focus is away from a form element
change	onchange	When the user modifies or changes the value of a form element

Window/Document events

Event Performed	Event Handler	Description
-----------------	---------------	-------------

load	onload	When the browser finishes the loading of the page
unload	onunload	When the visitor leaves the current webpage, the browser unloads it
resize	onresize	When the visitor resizes the window of the browser

Examples –

```

<html>
<head> Javascript Events </head>
<body>
<script language="Javascript" type="text/Javascript">
    function clickevent()
    {
        document.write("This is JavaTpoint");
    }
</script>
<form>
<input type="button" onclick="clickevent()" value="Who's this?"/>
</form>
</body>
</html>

```

O/P

This is JavaTpoint Javascript Events

After we click, we get -

This is JavaTpoint

Example 2 –

```

<html>
<head>
<h1> Javascript Events </h1>
</head>
<body>
<script language="Javascript" type="text/Javascript">
    function mouseoverevent()
    {
        alert("This is JavaTpoint");
    }

```

```
</script>
<p onmouseover="mouseoverevent()"> Keep cursor over me</p>
</body>
</html>
```

O/P

Javascript Events

Keep cursor over me

After mouseover



Example –

```
<html>
<head>Javascript Events</head>
<br>
<body onload="window.alert('Page successfully loaded');">
<script>
<!--
document.write("The page is loaded successfully");
//-->
</script>
</body>
</html>
```

JavaScript objects

https://www.tutorialspoint.com/javascript/javascript_objects.htm

A JavaScript object is an entity having state and behavior (properties and method). For example: car, pen, bike, chair, glass, keyboard, monitor etc.

JavaScript is an object-based language. Everything is an object in JavaScript.

JavaScript is template based not class based. Here, we don't create class to get the object. But, we directly create objects.

Objects are composed of attributes. If an attribute contains a function, it is considered to be a method of the object, otherwise the attribute is considered a property.

Creating Objects in JavaScript

There are 3 ways to create objects.

1. By object literal
2. By creating instance of Object directly (using new keyword)
3. By using an object constructor (using new keyword)

1) JavaScript Object by object literal

The **syntax** of creating object using object literal is given below:

```
object={property1:value1,property2:value2.....propertyN:valueN}
```

As you can see, property and value is separated by : (colon).

Example –

```
<script>  
emp={id:102,name:"Shyam Kumar",salary:40000}  
document.write(emp.id+" "+emp.name+" "+emp.salary);  
</script>
```

Output

```
102 Shyam Kumar 40000
```

2) By creating instance of Object

The **syntax** of creating object directly is given below:

```
var objectname=new Object();
```

Here, **new keyword** is used to create object.

Example –

```
<script>  
var emp=new Object();  
emp.id=101;  
emp.name="Ravi Malik";
```

```
emp.salary=50000;  
document.write(emp.id+" "+emp.name+" "+emp.salary);  
</script>
```

Output

```
101 Ravi 50000
```

3) By using an Object constructor

Here, you need to create function with arguments. Each argument value can be assigned in the current object by using this keyword. The **this keyword** refers to the current object.

Example –

```
<script>  
function emp(id,name,salary){  
  this.id=id;  
  this.name=name;  
  this.salary=salary;  
}  
e=new emp(103,"Vimal Jaiswal",30000);  
  
document.write(e.id+" "+e.name+" "+e.salary);  
</script>
```

Output

```
103 Vimal Jaiswal 30000
```

Defining method in JavaScript Object

We can define method in JavaScript object. But before defining method, we need to add property in the function with same name as method.

Example -

```
<script>  
function emp(id,name,salary){  
  this.id=id;
```

```

this.name=name;
this.salary=salary;

this.changeSalary=changeSalary;
function changeSalary(otherSalary){
this.salary=otherSalary;
}
}

e=new emp(103,"Sonoo Jaiswal",30000);
document.write(e.id+" "+e.name+" "+e.salary);
e.changeSalary(45000);
document.write("<br>" +e.id+" "+e.name+" "+e.salary);
</script>

```

Output

```

103 Sonoo Jaiswal 30000
103 Sonoo Jaiswal 45000

```

7.8. Document Object Model in JavaScript

https://www.tutorialspoint.com/javascript/javascript_html_dom.htm

What is the DOM?

The DOM is a W3C (World Wide Web Consortium) standard.

The DOM defines a standard for accessing documents:

"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."

The W3C DOM standard is separated into 3 different parts:

- Core DOM - standard model for all document types
- XML DOM - standard model for XML documents
- HTML DOM - standard model for HTML documents

What is the HTML DOM?

The HTML DOM is a standard **object** model and **programming interface** for HTML. It defines:

- The HTML elements as **objects**

- The **properties** of all HTML elements
- The **methods** to access all HTML elements
- The **events** for all HTML elements

In other words: **The HTML DOM is a standard for how to get, change, add, or HTML elements.**

The **document object** represents the whole html document.

When html document is loaded in the browser, it becomes a **document object**. It is the **root element** that represents the html document. It has properties and methods. By the help of document object, we can add dynamic content to our web page.

- With the HTML DOM, JavaScript can access and change all the elements of an HTML document.
- When a web page is loaded, the browser creates a **Document Object Model** of the page.
- The **HTML DOM** model is constructed as a tree of **Objects**:

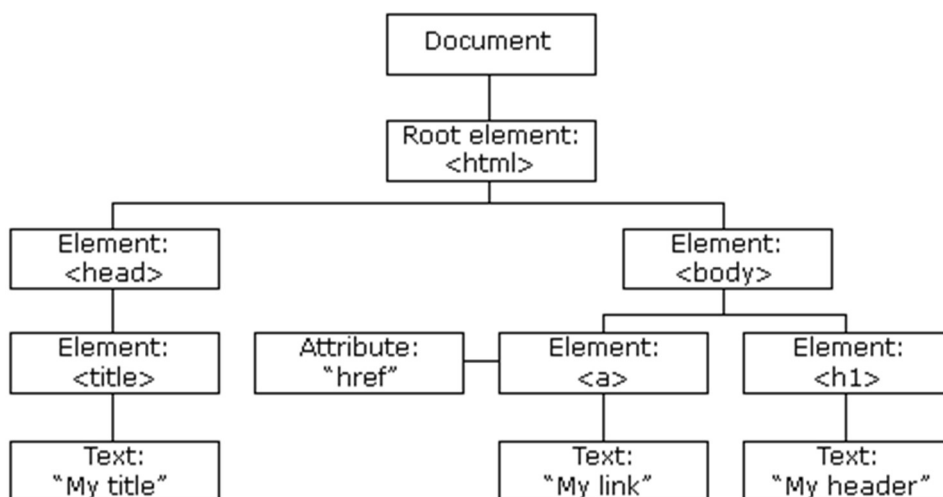


Fig - The HTML DOM Tree of Objects

With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page.
- JavaScript can change all the HTML attributes in the page.
- JavaScript can change all the CSS styles in the page.
- JavaScript can remove existing HTML elements and attributes.
- JavaScript can add new HTML elements and attributes.
- JavaScript can react to all existing HTML events in the page.
- JavaScript can create new HTML events in the page.

Example –

```

<script type="text/javascript">
function printvalue(){

```



```
var name=document.form1.name.value;  
alert("Welcome: "+name);  
}  
</script>
```

```
<form name="form1">  
Enter Name:<input type="text" name="name"/>  
<input type="button" onclick="printvalue()" value="print name"/>  
</form>
```

O/P



This is the simple example of document object that prints name with welcome message.

In this example, we get the value of input text by user. Here, we use **document.form1.name.value** to get the value of name field.

Here, **document** is the root element that represents the html document.

form1 is the name of the form.

name is the attribute name of the input text.

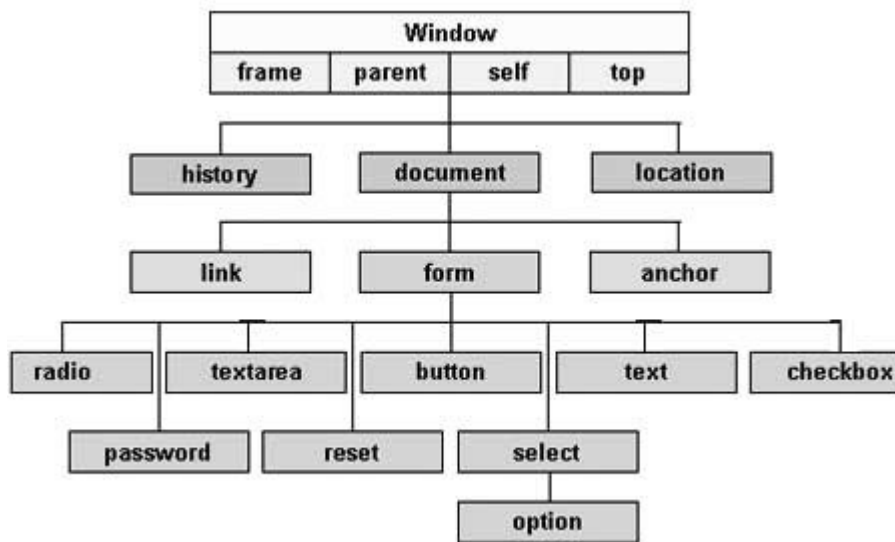
value is the property, that returns the value of the input text.

Note –

The way a document content is accessed and modified is called the **Document Object Model**, or **DOM**. The Objects are organized in a hierarchy. This hierarchical structure applies to the organization of objects in a Web document.

- **Window object** – Top of the hierarchy. It is the outmost element of the object hierarchy.
- **Document object** – Each HTML document that gets loaded into a window becomes a document object. The document contains the contents of the page.
- **Form object** – Everything enclosed in the <form>...</form> tags sets the form object.
- **Form control elements** – The form object contains all the elements defined for that object such as text fields, buttons, radio buttons, and checkboxes.

Here is a simple hierarchy of a few important objects –



7.8.1. Browser Objects and Events

7.8.2. Document Objects and Events

7.8.3. Form Objects and Events

Assignment -

https://www.w3schools.com/js/js_ex_browser.asp

Window/Document events

Event Performed	Event Handler	Description
load	onload	When the browser finishes the loading of the page
unload	onunload	When the visitor leaves the current webpage, the browser unloads it
resize	onresize	When the visitor resizes the window of the browser

Example -

Load event

```

<html>
<head>Javascript Events</head>
</br>
<body onload="window.alert('Page successfully loaded');">

```

```
<script>
document.write("The page is loaded successfully");
</script>
</body>
</html>
```

Form events:

Event Performed	Event Handler	Description
focus	onfocus	When the user focuses on an element
submit	onsubmit	When the user submits the form
blur	onblur	When the focus is away from a form element
change	onchange	When the user modifies or changes the value of a form element

Example –

Focus Event

```
<html>
<head> Javascript Events</head>
<body>
<h2> Enter something here</h2>
<input type="text" id="input1" onfocus="focusevent()"/>
<script>

function focusevent()
{
    document.getElementById("input1").style.background=" aqua";
}

</script>
</body>
</html>
```

O/P

Enter something here

7.9. Dialog Box supported by JavaScript.

JavaScript supports three important types of dialog boxes. These dialog boxes can be used to raise an alert, or to get confirmation on any input or to have a kind of input from the users.

Alert Dialog Box, Confirm Dialog Box, Prompt Dialog Box.

Alert Dialog Box

An alert dialog box is mostly used to give a warning message to the users. For example, if one input field requires to enter some text but the user does not provide any input, then as a part of validation, you can use an alert box to give a warning message.

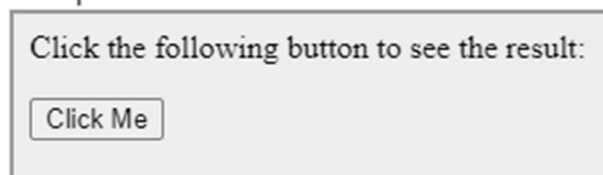
Nonetheless, an alert box can still be used for friendlier messages. Alert box gives only one button "OK" to select and proceed.

Example –

```
<html>
<head>
  <script type = "text/javascript">
    function Warn() {
      alert ("This is a warning message!");
      document.write ("This is a warning message!");
    }
  </script>
</head>

<body>
  <p>Click the following button to see the result: </p>
  <form>
    <input type = "button" value = "Click Me" onclick = "Warn();" />
  </form>
</body>
</html>
```

Output



After we click "Click Me" button

www.tutorialspoint.com says

This is a warning message!

OK

Confirmation Dialog Box

A confirmation dialog box is mostly used to take user's consent on any option. It displays a dialog box with two buttons: **OK** and **Cancel**.

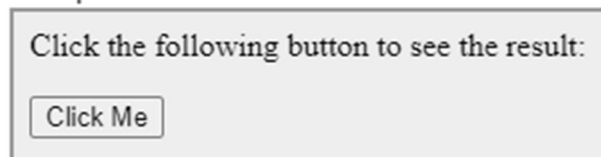
If the user clicks on the OK button, the window method **confirm()** will return true. If the user clicks on the Cancel button, then **confirm()** returns false. You can use a confirmation dialog box as follows.

Example –

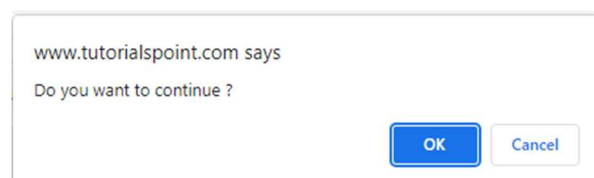
```
<html>
<head>
  <script type = "text/javascript">
    function getConfirmation() {
      var retVal = confirm("Do you want to continue?");
      if( retVal == true ) {
        document.write ("User wants to continue!");
        return true;
      } else {
        document.write ("User does not want to continue!");
        return false;
      }
    }
  </script>
</head>

<body>
  <p>Click the following button to see the result: </p>
  <form>
    <input type = "button" value = "Click Me" onclick = "getConfirmation();" />
  </form>
</body>
</html>
```

Output



After we click "Click Me" button



Prompt Dialog Box

The prompt dialog box is very useful when you want to pop-up a text box to get user input. Thus, it enables you to interact with the user. The user needs to fill in the field and then click OK.

This dialog box is displayed using a method called **prompt()** which takes two parameters: (i) a label which you want to display in the text box and (ii) a default string to display in the text box.

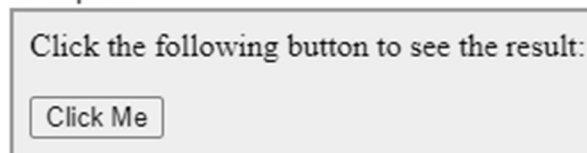
This dialog box has two buttons: **OK** and **Cancel**. If the user clicks the OK button, the window method **prompt()** will return the entered value from the text box. If the user clicks the Cancel button, the window method **prompt()** returns **null**.

Example –

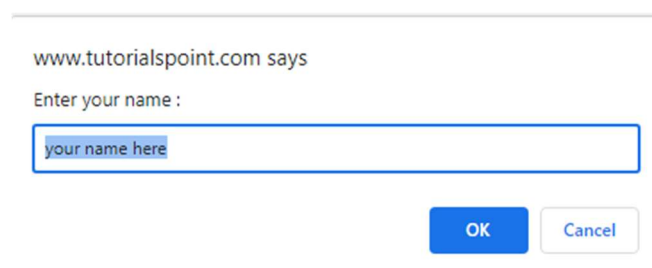
```
<html>
<head>
  <script type = "text/javascript">
    function getValue() {
      var retVal = prompt("Enter your name : ", "your name here");
      document.write("You have entered : " + retVal);
    }
  </script>
</head>

<body>
  <p>Click the following button to see the result: </p>
  <form>
    <input type = "button" value = "Click Me" onclick = "getValue();" />
  </form>
</body>
</html>
```

Output



After we click “Click Me” button



7.10. Form validation

https://www.tutorialspoint.com/javascript/javascript_form_validations.htm

<https://www.javatpoint.com/javascript-form-validation>

It is important to validate the form submitted by the user because it can have inappropriate values. So, validation is must to authenticate user.

JavaScript provides facility to validate the form on the client-side so data processing will be faster than server-side validation. Most of the web developers prefer JavaScript form validation.

Through JavaScript, we can validate name, password, email, date, mobile numbers and more fields.

Example –

Here, we are validating the form on form submit. The user will not be forwarded to the next page until given values are correct.

```
<script>
function validateform(){
var name=document.myform.name.value;
var password=document.myform.password.value;

if (name==null || name==""){
    alert("Name can't be blank");
    return false;
}else if(password.length<6){
    alert("Password must be at least 6 characters long.");
    return false;
}
}
</script>
<body>
<form name="myform" method="post" action="abc.jsp" onsubmit="return validateform()" >
Name: <input type="text" name="name"> <br/>
Password: <input type="password" name="password"> <br/>
<input type="submit" value="register">
</form>
```